

RESULTS

A2A (Google Agent2Agent) — LIVE Re-validation with the REAL a2a-go SDK, RESULTS

Run-id	a2a_live_e2e_20260711T134958Z
Track	(T1/feature/helixllm-full-extension)
Scope	Re-validate the HelixLLM A2A server (proven at helix_llm commit 6e21dde, prior evidence docs/qa/phase3_a2a_20260707/) LIVE, using the real upstream github.com/a2aproject/a2a-go SDK v0.3.15 as the client — never curl-faking / hand-rolled JSON-RPC — completing the API-surface V&V
Verdict	DONE, with two honest findings — Agent Card real, real Task nonce-echoed by the live Qwen coder (confirmed via real-SDK tasks/get round-trip + raw wire evidence), JSON-RPC method strings match the v0.3.0 spec binding exactly; two genuine spec-conformance gaps in the server's wire shape were surfaced by the real SDK (never present in the earlier hand-rolled harness) and are documented below, not silently worked around in the server (§11.4.122 read-only-on-submodule)
Repo (main)	see git log -1 at commit time (below)
Submodule (helix_llm)	untouched except gitignored bin/a2a-server build output — no source edit

1. Constraints respected

- Coder : 18434 (llama-server, PID 1980342, model Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf) — **read-only, never restarted**. Confirmed reachable (HTTP 200 on /v1/models) before, during (via the A2A round trip), and after this session.
 - A2A server built + started on the distinct port : 18441 (harness ports 18436–18440 are other streams' — untouched).
 - submodules/helix_llm — **zero source edits**. Only go build -o bin/a2a-server ./cmd/a2a-server was run (gitignored output, bin/ per .gitignore:10) and go test ./internal/a2a/... (read-only). An unrelated untracked file belonging to a different concurrent stream (cmd/agentgen-boot/agentgen-boot) was left untouched (§11.4.119 / §11.4.174).
 - All new harness code + captured evidence lives at the **repo ROOT** under docs/qa/a2a_live_e2e_20260711T134958Z/ — no submodule contention with the RAG/CPU-caps streams.
 - Real a2a-go SDK client only (github.com/a2aproject/a2a-go@v0.3.15, fetched live from proxy.golang.org — §11.4.99 latest-source) — never a hand-rolled JSON-RPC/curl fake. The harness is a genuinely black-box A2A peer: it never imports HelixLLM's internal/a2a package.
-

2. What was proven

2.1 Server started, real Agent Card

```
2026/07/11 18:50:21 discovered downstream model id="/models/Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf"
2026/07/11 18:50:21 A2A server listening on :18441 (base-path=/a2a public-path=/a2a)
(a2a_server.log)
```

agentcard.DefaultResolver.Resolve(ctx, "http://localhost:18441")
— the REAL SDK's own resolver — fetched 01_agent_card.json. Decoded into a real *a2a.AgentCard struct with every field populated and checked:

Check	Result
name non-empty (helixllm-coder-a2a)	PASS
description non-empty	PASS
url non-empty	PASS
skills[] non-empty	PASS
skill id=="generate-code" present	PASS

Check	Result
<code>preferredTransport == "JSONRPC"</code> (matches <code>a2a.TransportProtocolJSONRPC</code>)	PASS
<code>securitySchemes.bearer</code> present, decodes as <code>a2a.HTTPAuthSecurityScheme{Type:"http",Scheme:"bearer"}</code> via the real SDK's discriminated-union <code>NamedSecuritySchemes.UnmarshalJSON</code>	PASS

Agent Card is **real** — not a stub, not a fixture; it is the live `bin/a2a-server` process's actual `GET /.well-known/agent-card.json` response, decoded end-to-end by the real SDK.

2.2 Real Task sent via the real a2a-go SDK client → live coder nonce-echo

A fresh nonce (`NONCE_dad0790a23f07218`) was embedded in a real prompt and sent via `a2aclient.Client.SendMessage()` (constructed via `a2aclient.NewFromEndpoints` + the real JSON-RPC transport + a real Bearer-token `CallInterceptor` — `a2aclient.NewStaticCallMetaInjector`, a first-class SDK primitive, not a hand-rolled header hack). The capturing transport (a real `http.RoundTripper` wrapper around `http.DefaultTransport` — genuine TCP/HTTP, nothing faked) recorded the raw wire exchange to `wire_02_message_send.txt`:

```
{"jsonrpc":"2.0","method":"message/send","params":{"message":
  {"kind":"message","messageId":"...","parts":[{"kind":"text","text"
with ONLY this exact token and nothing else:
  NONCE_dad0790a23f07218"}],"role":"user"}}, "id":"..."}
```

→ HTTP 200 →

```
{"id":"4bf1e5a7-6045-4f34-be00-4ee0a1d69922","status":{"state":"completed"
  [{"kind":"text","text":"NONCE_dad0790a23f07218"}]}],"artifacts":[{"
```

The live Qwen3-Coder-30B model genuinely answered with the exact nonce echoed back (`raw_artifact_contains_nonce: true`, `99_report.json`) — this is real model output over the real `:18434 /v1/chat/completions` route, routed through the real A2A server, round-tripped through the real a2a-go SDK's HTTP transport. It is impossible to fake this signature with a stub handler (a fresh, per-run random nonce that only the live model saw in the prompt).

2.3 tasks/get round-trip via the REAL SDK — full success

```
[4] client.GetTask (tasks/get) round-trip on id=4bf1e5a7-6045-4f34-be00-4e
  client.GetTask via REAL SDK: id_matches=true state=completed nonce_pre
```

Unlike `SendMessage` (§2.4 below), `client.GetTask()` uses a direct (non-polymorphic) `json.Unmarshal(result, &task)` in the SDK's `jsonrpcTransport.GetTask` — this call **succeeded cleanly** through the real SDK's typed API: the returned `*a2a.Task` has the identical id, `state=="completed"`, and the nonce present in its artifact text (`wire_03_tasks_get.txt`). This is the clean, fully-real-SDK-typed proof of the Task round-trip.

2.4 Two genuine spec-conformance findings (discovered ONLY by using the real SDK)

The prior hand-rolled harness (`docs/qa/phase3_a2a_20260707/harness/main.go`) parsed responses with its own permissive ad-hoc Go structs and never caught either of these — they only surface when a **real, spec-faithful reference client** is used, which is exactly the point of this re-validation pass.

Finding A — `AgentCard.url` omits the JSON-RPC dispatch base path. The server's Agent Card declares `"url": "http://localhost:18441"` (no `/a2a` suffix), but the actual JSON-RPC dispatch route is mounted at `POST /a2a` (`internal/a2a/router.go:39`). Per the A2A spec's own binding (mirrored verbatim in the vendored SDK's doc comment, `a2a-go@v0.3.15/a2a/agent.go:84-89`): *"PreferredTransport is the transport protocol for the preferred endpoint (the main 'url' field) ... IMPORTANT: The transport specified here MUST be available at the main 'url' field."* `a2aclient.NewFromCard` builds its sole transport candidate as `{Transport: card.PreferredTransport, URL: card.URL}` — i.e. it POSTs JSON-RPC requests directly to `card.URL`. Doing exactly that against the live server returned **HTTP 404** (`wire_01_message_send.txt`, `02_card_url_conformance_probe.json`). A real, spec-faithful A2A client that trusts the Agent Card literally cannot reach this server's JSON-RPC endpoint. - **Not fixed in this pass** — `submodules/helix_llm` is read-only for this task (§11.4.122 / stream-isolation instruction). The harness worked around this ONLY on the client side (§2.2/§2.3, `a2aclient.NewFromEndpoints` with an explicit `.../a2a` URL) to still complete the deeper round-trip proof; the server-side Agent Card is unchanged. - **Fix direction for the owning stream:** set `card.URL = cfg.PublicURL + basePath` (i.e. include `/a2a`) in `internal/a2a/agentcard.go`'s `BuildAgentCard`, or declare `AdditionalInterfaces` covering the actual dispatch path.

Finding B — the server's message/send result (a Task) lacks the top-level "kind": "task" discriminator the real SDK requires.

`a2aclient.Client.SendMessage()` decodes its result via `a2a.UnmarshalEventJSON`, which reads a `"kind"` field first to decide whether the JSON is a Task or a Message (`a2a-go@v0.3.15/a2a/core.go:87-124`; the real SDK's own `Task.MarshalJSON` always injects `{"kind": "task", ...}`). `HelixLLM`'s `internal/a2a/types.go` Task struct has no `Kind/kind` field, so the

server's message/send response is missing it. Against the live server, `client.SendMessage()` returned:

```
result violates A2A spec - could not determine type: unknown event kind: ;
```

even though the HTTP transaction and the coder's answer (§2.2) were completely genuine and correct. Sub-object Parts (inside `Artifacts[] .Parts[] / History[] .Parts[]`) DO carry `"kind": "text"` correctly (per `internal/a2a/types.go Part{Kind: PartKindText, ...}`) — only the top-level Task/Message envelope is missing its own discriminator. - **Not fixed in this pass** — same read-only constraint. The RAW wire evidence (§2.2) plus the `tasks/get` round-trip (§2.3, which does not require `kind`) independently prove the coder-nonce-echo signature despite this client-side typed-decode gap. - **Fix direction for the owning stream:** add `Task.MarshalJSON` (or a `Kind string \json:"kind"` field defaulted to `"task"`) to `internal/a2a/types.go`, mirroring the real SDK's `ownTask.MarshalJSON/Message.MarshalJSON` pattern.

Both findings are genuine, reproducible, and were captured with real over-the-wire evidence — not guessed, not asserted from memory (§11.4.6). Neither was silently worked around inside `submodules/helix_llm`.

2.5 JSON-RPC method-string ↔ spec v0.3.0 conformance — full match

The vendored real SDK's own wire-protocol constants (`a2a-go@v0.3.15/internal/jsonrpc/jsonrpc.go`):

```
MethodMessageSend = "message/send"
MethodTasksGet    = "tasks/get"
```

match **exactly**, byte-for-byte, the method strings HelixLLM's `internal/a2a/handlers.go DispatchHandler` switches on (`case "message/send":`, `case "tasks/get":`). This was confirmed two independent ways: (a) direct source comparison against the vendored `a2a-go@v0.3.15` module (fetched live, §11.4.99), and (b) empirically — the real SDK's own request-construction code (`jsonrpcTransport.newHTTPRequest`) produced the wire bytes `"method": "message/send" / "method": "tasks/get"` captured in `wire_02_message_send.txt / wire_03_tasks_get.txt`, and the live server correctly routed both. **No discrepancy at the method-string layer** — this is the “Rev2 correction” the design references, and it holds.

2.6 Coder untouched

Check	Before	After
llama-server PID	1980342	1980342 (unchanged)

Check	Before	After
GET /v1/models HTTP status	200	200

coder_after.json / coder_final.json — identical model metadata both times. The coder was never signalled, restarted, or reconfigured (§11.4.119 / §11.4.122).

2.7 Teardown

Only our own bin/a2a-server PID (2723249) was killed, verified by ps argv (. / bin/a2a-server) immediately before the kill (§11.4.174 process-ownership verification). Port :18441 confirmed free afterward. See 16_teardown.txt.

3. Honest scope (§11.4.6 / §11.4.101 / §11.4.143)

Per the design’s own honest-scope note (carried over unchanged from docs/qa/phase3_a2a_20260707/RESULTS.md) and re-confirmed by this pass:

- **message/stream (SSE) is NOT implemented or exercised** — the Agent Card truthfully declares capabilities.streaming=false; this session did not attempt SendStreamingMessage.
- **Push notifications are NOT implemented** — capabilities.pushNotifications=false, declared honestly, not exercised.
- **The extended/authenticated Agent Card flow is NOT implemented** — capabilities.extendedAgentCard=false.
- **gRPC / HTTP+JSON-REST transport bindings are NOT implemented** — only JSON-RPC 2.0-over-HTTP was exercised (the real SDK’s WithGRPCTransport default was present in the client factory but never selected, since only a JSONRPC AgentInterface was offered).
- **File/Data Parts are not exercised** — only Text Parts were sent/asserted.
- This is the **server** side only (HelixLLM reachable BY other A2A agents) — no A2A **client** capability (HelixLLM reaching OUT to other A2A peers) was built or exercised.
- The two spec-conformance findings in §2.4 are genuine open gaps, not silently worked around in the server — they are the “honest scope” finding of THIS re-validation pass specifically.

None of the above are silently dropped: each is a truthful false capability flag or an explicit statement here, never a faked PASS.

4. Reproduce

```
# 1. Build (no source edits – gitignored output)
cd submodules/helix_llm
go build -o bin/a2a-server ./cmd/a2a-server
go test -v -count=1 ./internal/a2a/...

# 2. Start the A2A server against the live coder (never restart
    the coder)
TOKEN=$(openssl rand -hex 16)
HELIX_A2A_LISTEN_ADDR=:18441 HELIX_A2A_DOWNSTREAM_URL=http://
    localhost:18434 \
    HELIX_A2A_BEARER_TOKEN="$TOKEN" ./bin/a2a-server &

# 3. Build + run the real-a2a-go-SDK harness
cd ../../docs/qa/a2a_live_e2e_20260711T134958Z/harness
go build -o a2a_live_e2e.bin .
./a2a_live_e2e.bin -base-url http://localhost:18441 -token
    "$TOKEN" -evidence-dir ..

# 4. Verify coder unaffected, then tear down only the a2a-server
    PID
curl -s http://localhost:18434/v1/models | head -c 200
kill %1 # or the a2a-server PID specifically
```

5. Sources verified (§11.4.99 / §11.4.150)

- <https://a2a-protocol.org/latest/specification/> (unchanged since docs / research/07.2026/00_master/ACP_A2A_PROVIDER.md's original citation, 2026-07-07)
 - <https://a2a-protocol.org/v0.3.0/specification/>
 - <https://github.com/a2aproject/A2A>
 - <https://github.com/a2aproject/a2a-go> — **the reference SDK itself, fetched live from proxy.golang.org at v0.3.15** (the latest v0.3.x release as of this session; v1.0.0-alpha.3 also exists upstream but the v0.3.x line matches the design's cited v0.3.0 spec binding) — used as the client in this pass, and its internal/jsonrpc, a2a/core.go, a2a/agent.go, a2a/auth.go source was read directly (not from training-data memory) to confirm method strings, envelope shapes, and the two findings in §2.4.
-

6. Composition footer

Composes §11.4.6 (honest scope; findings backed by captured evidence, never guessed) / §11.4.5 + §11.4.69 (positive downstream evidence — real nonce echoed by the live coder) / §11.4.99 + §11.4.150 (latest-source, the real SDK fetched live and read directly) / §11.4.108 (four-layer: SOURCE unchanged → ARTIFACT bin/a2a-server builds clean, 5/5 unit tests → RUNTIME-ON-LIVE-TARGET the nonce-echo + tasks/get signatures above → USER-VISIBLE a real SDK-driven peer gets real generated content back) / §11.4.119 + §11.4.122 (coder never touched/restarted, read-only downstream) / §11.4.174 (process-ownership verified by argv before the teardown kill) / §11.4.143 (real end-to-end capability round-trip, not a routing-only or sample shortcut — a fresh nonce is proof the specific request was genuinely processed by the live model, not a cached/canned reply).